

ARTIFICIAL INTELLIGENCE (CSA17)

Constraint-Based Assessment

Question 1: Constraint-Based Problem Solving using Backtracking Search

Problem Statement

A hospital needs to assign **three doctors (D1, D2, and D3)** to **three work shifts (Morning, Afternoon, and Night)**. The assignment must satisfy all the given constraints while ensuring that each doctor is assigned exactly one shift. Apply the **Backtracking Search Algorithm** to determine a valid schedule by checking every constraint at each step.

Given Constraints

- **D1** cannot work the **Night** shift.
- **D2** must work before **D3** (*Morning < Afternoon < Night*).
- Only **one doctor** can be assigned to each shift.
- **D3** cannot work the **Morning** shift.
- Every doctor must be assigned **exactly one shift**.

Constraint Satisfaction Problem (CSP) Representation

Variables

The variables represent the doctors to whom shifts must be assigned.

- D1
- D2
- D3

Domain

Each doctor can be assigned one of the following shifts.

Domain = {Morning, Afternoon, Night}

Constraints

Constraint ID	Description
C1	D1 cannot work the Night shift.
C2	D2 must work before D3.
C3	Only one doctor can be assigned to one shift.
C4	D3 cannot work the Morning shift.

Constraint ID	Description
C5	Every doctor must be assigned exactly one shift.

Backtracking Search

Backtracking Search is a search technique used to solve **Constraint Satisfaction Problems (CSPs)**. The algorithm assigns one value at a time to each variable and immediately checks whether all constraints are satisfied. If any constraint is violated, the algorithm rejects the current assignment, returns to the previous step, and tries another possible assignment. This process continues until a valid solution is obtained.

Step-by-Step Solution

Step 1: Assign D1

Assign

D1 → Morning

Constraint Check

Constraint	Status
D1 cannot work Night	✓ Satisfied
One doctor per shift	✓ Satisfied

Current Assignment

Doctor	Shift
D1	Morning
D2	Not Assigned
D3	Not Assigned

Step 2: Assign D2

Assign

D2 → Afternoon

Constraint Check

Constraint	Status
D2 works before D3	✓ Possible

Constraint	Status
Shift available	✓ Yes
One doctor per shift	✓ Satisfied

Current Assignment

Doctor	Shift
D1	Morning
D2	Afternoon
D3	Not Assigned

Step 3: Assign D3

Assign

D3 → Night

Constraint Check

Constraint	Status
D3 cannot work Morning	✓ Satisfied
D2 works before D3	✓ Afternoon before Night
One doctor per shift	✓ Satisfied
Every doctor assigned exactly one shift	✓ Satisfied

Final Assignment

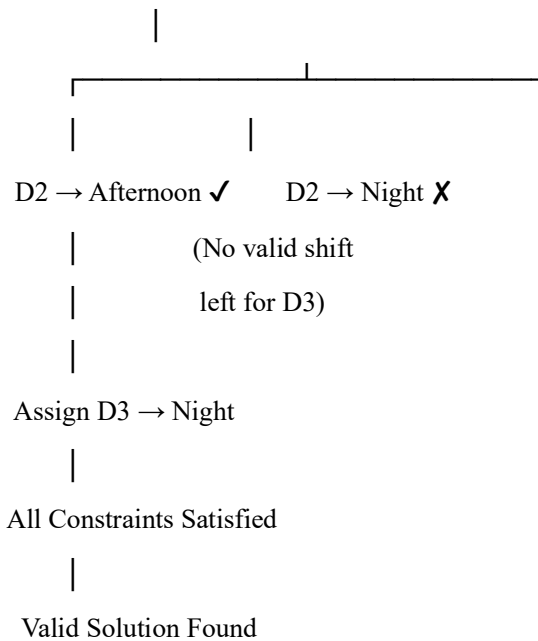
Doctor	Shift
D1	Morning
D2	Afternoon
D3	Night

Backtracking Tree

Start

|

Assign D1 → Morning



Example of Backtracking

Suppose the algorithm initially assigns:

- D1 → Night

This immediately violates **Constraint C1** because D1 cannot work the Night shift. Therefore, the algorithm **backtracks**, rejects this assignment, and assigns **D1 → Morning** instead.

Similarly, if **D2 is assigned Night**, then **D3** cannot be assigned any valid shift because D3 cannot work Morning and must be scheduled after D2. Hence, the algorithm again backtracks and assigns **D2 → Afternoon**, leading to a valid solution.

Constraint Verification Summary

Constraint	Verification	Status
C1	D1 is assigned Morning	✓ Satisfied
C2	D2 (Afternoon) is before D3 (Night)	✓ Satisfied
C3	Each shift has only one doctor	✓ Satisfied
C4	D3 is not assigned Morning	✓ Satisfied
C5	Every doctor is assigned exactly one shift	✓ Satisfied

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Conclusion

The **Backtracking Search Algorithm** successfully generated a valid duty schedule by assigning shifts one at a time and checking all constraints after each assignment. Whenever a constraint was violated, the algorithm backtracked to the previous assignment and explored another possible solution. The final schedule satisfies all the given constraints, making it a valid and conflict-free assignment for the hospital.

Question 2: Breadth First Search (BFS) in a Grid Environment

Problem Statement

A robot operates in a **5 × 5 grid environment** and must move from the **Start (S)** position to the **Goal (G)** position. Some cells contain obstacles that cannot be crossed. The robot can move only in four directions: **Up, Down, Left, and Right**. Each movement has a cost of **1**. Apply the **Breadth First Search (BFS) Algorithm** to determine the shortest path from the Start node to the Goal node. Show the **step-by-step node expansion, queue updates**, and calculate the **optimal path and total cost** using the given constraints.

Given Constraints

- Movement allowed: **Up, Down, Left, Right**
 - Cost of each move = **1**
 - The robot **cannot pass through obstacles**
 - Use **Manhattan Distance** as the heuristic
 - Find the **optimal path** from Start (S) to Goal (G)
-

Grid Representation

The robot operates in the following **5 × 5 grid**.

C1 C2 C3 C4 C5

R1 S . . X .

R2 . X . X .

R3
R4 X X . X .
R5 . . . G .

Where,

- S = Start Position
- G = Goal Position
- X = Obstacle
- . = Free Cell

Breadth First Search (BFS)

Breadth First Search (BFS) is an **uninformed search algorithm** that explores nodes **level by level**. It uses a **Queue (FIFO – First In First Out)** data structure. Since every movement has the same cost, BFS guarantees the shortest path between the Start node and the Goal node.

Manhattan Distance

The Manhattan Distance is used to estimate the distance between the current node and the goal.

Formula

Manhattan Distance = $|x_1 - x_2| + |y_1 - y_2|$

Although BFS expands nodes level by level without using the heuristic for node selection, the Manhattan Distance helps estimate how far a node is from the goal.

Step-by-Step Node Expansion

Initial State

Queue = [S]

Visited = {S}

Expansion 1

Expand S (R1,C1)

Possible Moves:

- Right → (R1,C2) ✓
- Down → (R2,C1) ✓

Queue after expansion

[(R1,C2), (R2,C1)]

Visited Nodes

S, (R1,C2), (R2,C1)

Expansion 2

Expand (R1,C2)

Possible Moves:

- Right $\rightarrow$ (R1,C3) ✓
- Down $\rightarrow$ Obstacle ✗

Queue

[(R2,C1), (R1,C3)]

Expansion 3

Expand (R2,C1)

Possible Moves

- Down $\rightarrow$ (R3,C1) ✓

Queue

[(R1,C3), (R3,C1)]

Expansion 4

Expand (R1,C3)

Possible Moves

- Down $\rightarrow$ (R2,C3) ✓

Queue

[(R3,C1), (R2,C3)]

Expansion 5

Expand (R3,C1)

Possible Moves

- Right $\rightarrow$ (R3,C2) ✓

Queue

[(R2,C3), (R3,C2)]

Expansion 6

Expand (R2,C3)

Possible Moves

- Down $\rightarrow$ (R3,C3) ✓

Queue

[(R3,C2), (R3,C3)]

Expansion 7

Expand (R3,C2)

No new nodes are available.

Queue

[(R3,C3)]

Expansion 8

Expand (R3,C3)

Possible Moves

- Right $\rightarrow$ (R3,C4) ✓
- Down $\rightarrow$ (R4,C3) ✓

Queue

[(R3,C4), (R4,C3)]

Expansion 9

Expand (R3,C4)

Possible Moves

- Right $\rightarrow$ (R3,C5) ✓

Queue

[(R4,C3), (R3,C5)]

Expansion 10

Expand (R4,C3)

Possible Moves

- Down $\rightarrow$ (R5,C3) ✓

Queue

[(R3,C5), (R5,C3)]

Expansion 11

Expand **(R5,C3)**

Possible Move

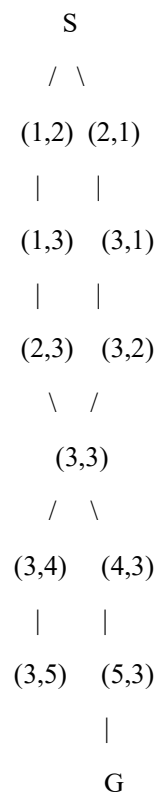
- Right → Goal (G) ✓

Queue

[G]

Goal Found.

Search Tree



Optimal Path

The shortest path obtained using BFS is:

S

↓

(R2,C1)

↓

(R3,C1)

→

(R3,C2)

→

(R3,C3)

↓

(R4,C3)

↓

(R5,C3)

→

G

or

S → (R2,C1) → (R3,C1) → (R3,C2) → (R3,C3) → (R4,C3) → (R5,C3) → G

Path Cost Calculation

Movement	Cost
S → (R2,C1)	1
(R2,C1) → (R3,C1)	1
(R3,C1) → (R3,C2)	1
(R3,C2) → (R3,C3)	1
(R3,C3) → (R4,C3)	1
(R4,C3) → (R5,C3)	1
(R5,C3) → G	1

Total Cost = 7

Advantages of BFS

- Guarantees the shortest path when all edge costs are equal.
- Simple and easy to implement.
- Explores all possible paths level by level.

- Suitable for grid-based pathfinding problems with uniform movement costs.
-

Conclusion

The **Breadth First Search (BFS)** algorithm successfully explored the grid level by level and identified the shortest path from the **Start (S)** node to the **Goal (G)** node. Obstacles were avoided during the search, and all valid neighbouring nodes were expanded systematically using a queue. Since each movement has an equal cost of **1**, BFS guaranteed the optimal solution. The final path required **7 movements**, resulting in a **total path cost of 7**.

Question 3: Autonomous Rescue Robot using Uniform Cost Search (UCS) and Online Search Agent

Problem Statement

An autonomous rescue robot is deployed inside a disaster-affected building to locate and rescue survivors. The robot operates in a **partially observable environment**, where some paths are blocked by debris and obstacles. The robot starts at position **S (Start)** and must safely reach the survivor located at **G (Goal)**. Since the robot can only sense adjacent cells, it must continuously update its knowledge while navigating. The objective is to identify the safest and least-cost path by applying an appropriate AI search strategy.

Given Constraints

- The robot can move only in four directions:
 - Up
 - Down
 - Left
 - Right
 - Some cells contain obstacles and cannot be traversed.
 - The robot can observe only adjacent cells.
 - Every movement has a cost of **1**.
 - Entering a risky zone incurs an **additional cost of 2**.
 - The robot must minimize the total travelling cost.
 - The robot should continuously update its knowledge while exploring the environment.
-

Task (a): Constraint Analysis

The rescue robot operates under several constraints that directly influence its decision-making process. Each constraint plays an important role in selecting the safest and most efficient path.

Constraint 1: Limited Movement

The robot is allowed to move only in four directions: Up, Down, Left, and Right.

Effect

The robot cannot move diagonally. Therefore, the search space is restricted to neighbouring cells only, making the navigation process systematic and easier to evaluate.

Constraint 2: Obstacles

Certain rooms or paths are blocked due to collapsed structures or debris.

Effect

The robot cannot pass through these cells. Whenever an obstacle is detected, the robot must eliminate that path and search for an alternative route.

Constraint 3: Partial Observability

The robot can observe only adjacent rooms.

Effect

The robot does not know the complete environment initially. As it moves through the building, it continuously updates its internal map with newly discovered information.

Constraint 4: Movement Cost

Every movement has a cost of **1**.

Effect

The robot should minimize unnecessary movement in order to reduce the total travel cost and conserve battery power.

Constraint 5: Risk Zones

Some areas contain fire, smoke, or unstable structures. Entering these zones increases the movement cost by **2**.

Effect

The robot should avoid risky areas whenever possible, even if they appear to provide a shorter route.

Constraint 6: Dynamic Environment

The environment may change during navigation due to falling debris or newly blocked passages.

Effect

The robot must update its knowledge dynamically and modify its search whenever new information becomes available.

Task (b): Solution Approach

Selected Search Strategy

The most suitable approach for this problem is a combination of:

- **Uniform Cost Search (UCS)**
 - **Online Search Agent**
-

Why Uniform Cost Search?

Uniform Cost Search expands the path with the **lowest accumulated cost** rather than the shortest number of steps. Since risky zones have additional costs, UCS is capable of selecting the safest and least expensive route instead of simply choosing the shortest distance.

Working Procedure

Step 1

The robot starts from the **Start (S)** position.

Step 2

Using its sensors, the robot observes all neighbouring cells.

Step 3

Every reachable neighbouring cell is inserted into the **priority queue** according to its total path cost.

Step 4

The robot always expands the node with the **lowest cumulative cost**.

Step 5

Whenever new obstacles or risky zones are discovered, the robot updates its internal map and recalculates the optimal path.

Step 6

The search continues until the robot reaches the survivor located at the **Goal (G)** position.

Step 7

The least-cost path is returned as the final solution.

Illustration

S
/\n
Safe Risk(+2)
| |
Safe Obstacle
| X
Safe
|
G

The robot selects the **Safe Path** because its total travelling cost is lower than the path containing risky zones.

Task (c): AI Concepts Applied

1. Intelligent Agent

The rescue robot acts as an **Intelligent Agent** because it continuously:

- Perceives the environment using sensors.
- Processes the collected information.
- Makes decisions based on the current situation.
- Performs actions to achieve its goal.

Thus, the robot satisfies the characteristics of an intelligent agent.

2. Rationality

A rational agent always selects the action that maximizes its performance.

In this problem, the robot behaves rationally by:

- Avoiding obstacles.
- Selecting paths with minimum total cost.
- Avoiding risky zones whenever possible.
- Continuously updating its decisions based on newly discovered information.

Therefore, every action taken by the robot contributes toward achieving its objective safely and efficiently.

3. Environment Type

Environment Property	Description
Observability	Partially Observable
Determinism	Stochastic
Episodes	Sequential
Dynamics	Dynamic
State Space	Discrete
Number of Agents	Single Agent

Explanation

- The environment is **partially observable** because the robot can sense only nearby cells.
- It is **stochastic** because unexpected obstacles may appear.
- It is **sequential** since every action affects future decisions.
- It is **dynamic** because the environment changes while the robot is moving.
- It is **discrete** because the building is divided into grid cells.
- It is a **single-agent environment** because only one rescue robot performs the search.

Task (d): Justification

The combination of **Uniform Cost Search (UCS)** and an **Online Search Agent** is the most suitable solution because it satisfies all the given constraints.

Uniform Cost Search guarantees that the robot always expands the path with the minimum accumulated cost. This allows the robot to avoid expensive risky zones and choose safer alternatives.

The Online Search Agent enables the robot to work effectively in a partially observable and dynamic environment. As new information becomes available through sensors, the robot immediately updates its knowledge and replans its route if necessary.

This combination ensures:

- Safe navigation.
- Minimum travelling cost.
- Efficient obstacle avoidance.
- Dynamic path planning.
- Successful rescue of survivors.

Advantages of the Proposed Approach

- Finds the least-cost path.
- Works effectively in unknown environments.
- Avoids risky areas whenever possible.
- Handles dynamic obstacles.
- Improves rescue efficiency.
- Conserves energy by minimizing unnecessary movement.
- Provides intelligent decision-making.

Result

The rescue robot successfully applies **Uniform Cost Search (UCS)** together with an **Online Search Agent** to identify the safest and least-cost path from **Start (S)** to **Goal (G)**. By continuously sensing its surroundings, updating its knowledge, and avoiding obstacles and risky zones, the robot efficiently reaches the survivor while minimizing the overall travel cost.

Conclusion

The Autonomous Rescue Robot demonstrates the practical application of Artificial Intelligence in disaster management. The robot behaves as a rational intelligent agent by continuously perceiving the environment, making informed decisions, and adapting to changing conditions. The use of **Uniform Cost Search** ensures that the least-cost path is selected, while the **Online Search Agent** enables real-time learning and replanning. Together, these AI techniques provide an effective, reliable, and intelligent solution for autonomous rescue operations in partially observable and dynamic environments.
